



COS6012-B: Concurrent and Distributed Systems

Processes and Threads

Dr Daniele Scrimieri

Outline

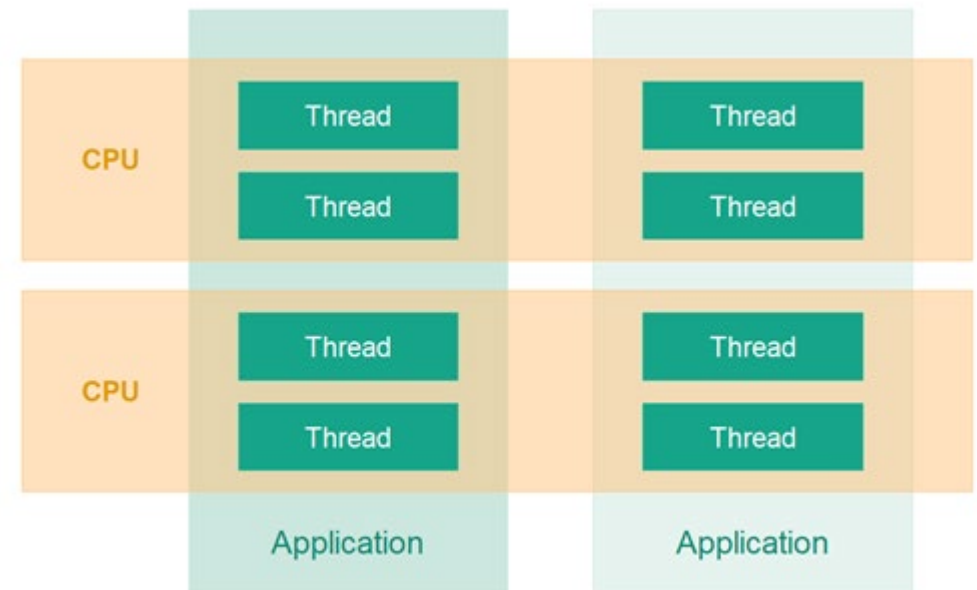
- Processes and threads
- Java concurrency and multithreading
- The lifecycle of a **Thread** in Java
- Creating and starting Java threads
- Useful methods
- Common pitfalls

Concurrent Processes

- In concurrent programming, there are two basic units of execution: **processes** and **threads**.
- Java concurrent programming is **mostly** concerned with **threads**.
- We structure **complex systems** as sets of simpler activities, each represented as a sequential **process**.
- Processes can overlap or be **concurrent**, to offload time-consuming tasks, or to manage communications or other devices.
- A process has a **self-contained execution environment**.
- A process generally has a complete, private set of basic run-time resources; in particular, each process has its own **memory** space.
- **Threads** are sometimes called **lightweight processes**.
- Threads exist within a process — every process has at least one.

Java Concurrency and Multithreading

- Java concurrency is a term that covers multithreading, concurrency and parallelism on the Java platform.
 - **Multithreading** means multiple threads of execution inside the same application.
-
- ✓ Better utilization of a single CPU.
 - ✓ Better utilization of multiple CPUs or CPU cores.
 - ✓ Better user experience with regards to **responsiveness**.
 - ✓ Better user experience with regards to **fairness**.



Java: Useful Classes and Packages for Concurrency

- Class **java.lang.Thread**: thread of execution in a program. JVM allows multiple threads running concurrently
- Package **java.util.concurrent**: utility classes commonly useful in concurrent programming, e.g. Semaphore, CyclicBarrier, ReadWriteLock, ForkAndJoin etc.

For distributed processing: threads in different JVMs communicate by message passing or remote procedure call

- Package **java.net** provides the classes for implementing networking applications: Addresses, Sockets, Interfaces, URUs, URLs, Connections.
- Package **java.rmi** - Java API that performs Remote Method Invocation (RMI), the object-oriented equivalent of remote procedure calls (RPC), with support for direct transfer of serialized Java classes and distributed garbage-collection.

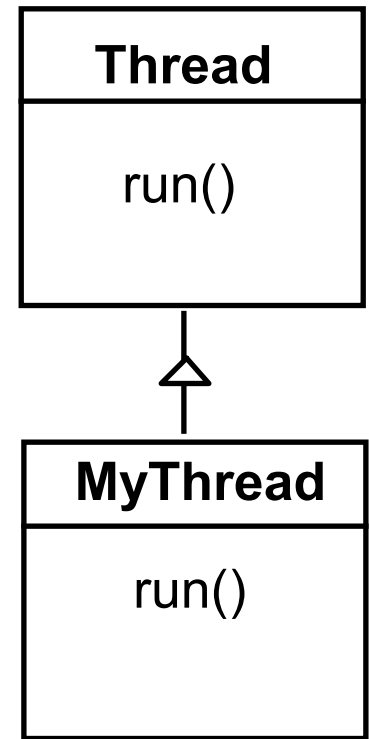
Threads in Java

- A thread is a **single sequential flow of control** within a Java program.
- Within the JVM, the threads comprising a Java program are represented by instances of the Thread class.
- Java threads are objects like any other Java objects.
- Threads are instances of class `java.lang.Thread`, or instances of subclasses of this class.
- In addition to being objects, Java threads can also execute code.
- There are two ways to create a thread:
 - **extending the Thread class** and overriding its `run()` method;
 - defining a class which **implements the Runnable interface** and its `run()` method and passing the Runnable object to the Thread constructor

Extending the Thread class

- Threads may be created and deleted dynamically.
- The Thread class executes instructions from its method run(). The actual code executed depends on the implementation provided for **run()** in a derived class.

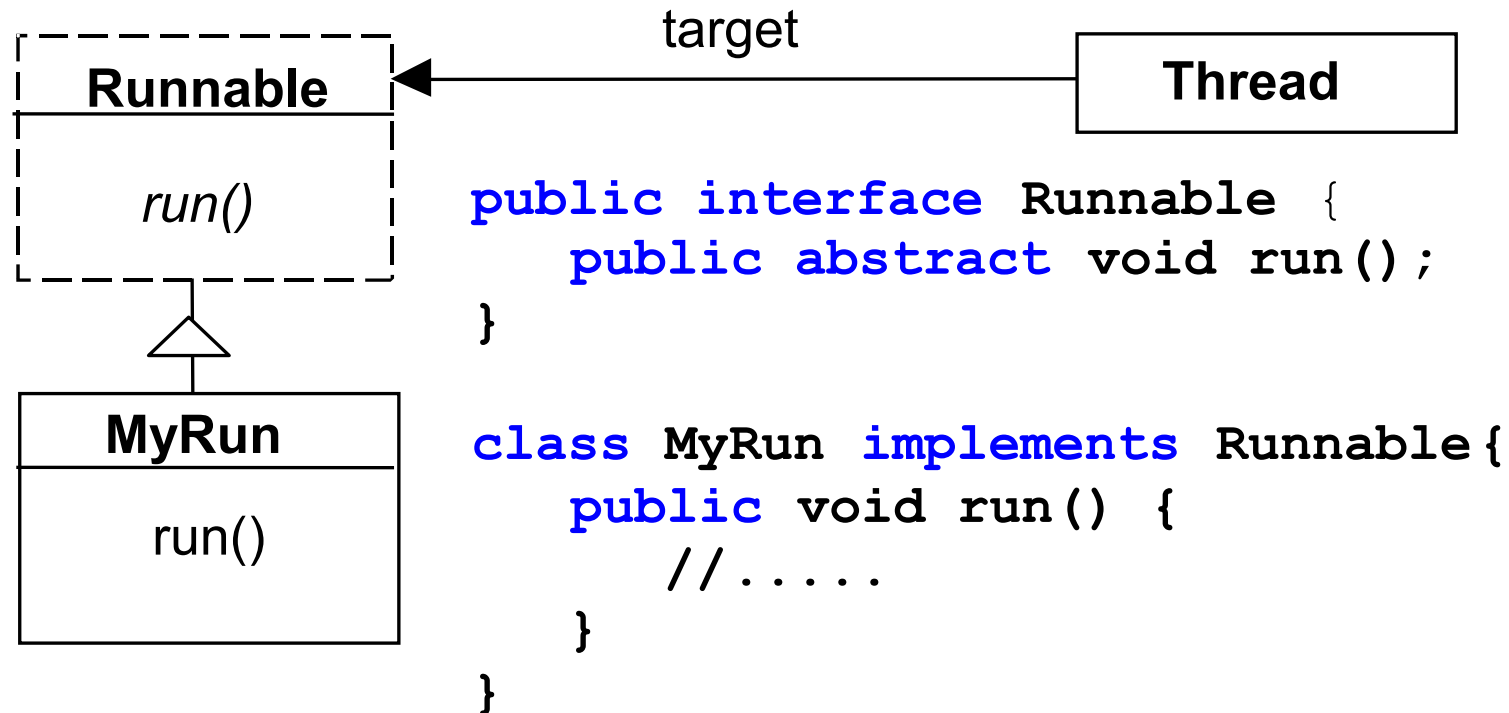
```
class MyThread extends Thread {  
    public void run() {  
        //.....  
    }  
}
```



Creating and starting a thread object:

```
Thread t1 = new MyThread();  
t1.start();
```

Implementing the Runnable Interface



Creating and starting a thread object:

```
Thread t2 = new Thread(new MyRun());  
t2.start();
```

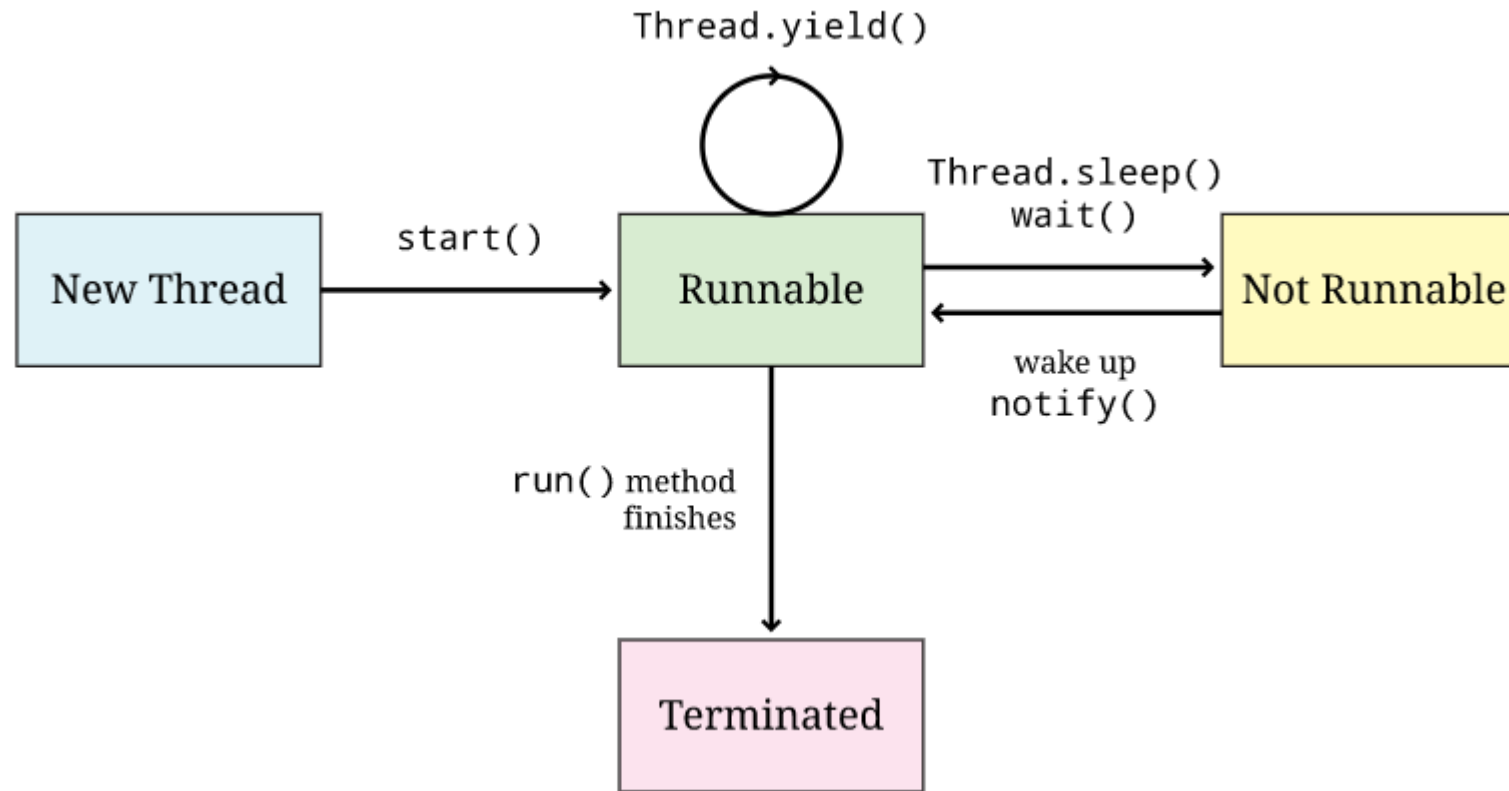

Thread Characteristics

- **ID:** This attribute stores a unique identifier for each thread.
- **Name:** This attribute stores the name of the thread.
- **Priority:** This attribute stores the priority of the Thread objects.
 - In Java 9, threads can have priority between MIN_PRIORITY=1 and MAX_PRIORITY=10
 - It's not recommended that you change the priority of the threads.
 - It's only a hint to the underlying operating system and it doesn't guarantee anything, but it's a possibility that you can use if you want. The threads have same default priority NORM_PRIORITY = 5
- **Status:** This attribute stores the status of a thread.
 - A thread can be present in one of the six states defined in the **Thread.State enumeration**: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, or TERMINATED.

Thread Life Cycle in Java

- **NEW:** The thread has been created, it has not yet started
- **RUNNABLE:** The thread is being executed in the JVM
- **BLOCKED:** The thread is blocked, it is waiting for a monitor
- **WAITING:** The thread is waiting for another thread
- **TIMED_WAITING:** The thread is waiting for another thread with a specified waiting time
- **TERMINATED:** The thread has finished its execution

Thread Life Cycle in Java



To simplify the figure, BLOCKED, WAITING, TIMED_WAITING are merged in the “Not Runnable” state.

Some Useful Methods

- **start()** Starts the thread.
- **getState()** Returns the state of the thread.
- **getName()** Returns the name of the thread.
- **getPriority()** Returns the priority of the thread.
- **sleep(long msec)** Suspend the thread for the specified milliseconds.
- **wait(), wait(...)** Wait indefinitely until another thread *notifies* it.
- **join()** Allows one thread to wait for the completion of another
- **yield()** Hint to scheduler: ready to yield processor.
- **notify(), notifyAll()** To wake up other threads that are waiting.
- **isAlive()** Check if the thread is alive.
- **isInterrupted()** Check whether the thread has been interrupted

Waiting for a Thread

- The **join()** method is used to wait for a thread to finish executing.
- For example, whichever thread executes the following code will wait for thread1 to complete.
- Sleep, join respond to an interrupt by exiting with an InterruptedException

```
try {  
    System.out.println("Waiting for thread 1...");  
    thread1.join();  
    System.out.println("Thread 1 finished!");  
}  
catch (InterruptedException e) {  
    System.out.println("Thread 1 didn't finish!");  
}
```

Simple Example with 2 Threads

- Will be using 2 threads to compute a mathematical expression

```
public class Thread1 extends Thread
{
    private double K, a, t, value;

    public Thread1(double K, double a,
        double t) {
        this.K = K;
        this.a = a;
        this.t = t;
    }

    public void run() {
        value = 2*K*a*t;
    }

    public double getValue() {
        return value;
    }
}
```

```
public class Thread2 extends
Thread
{
    private double a, t, value;

    public Thread2(double a,
        double t) {
        this.a = a;
        this.t = t;
    }

    public void run() {
        value = Math.exp(-a*t*t);
    }

    public double getValue() {
        return value;
    }
}
```

Main Class: Using the 2 Worker Threads

```
public class MathExpression {
    public static void main (String[] args) {
        double K = 120, a = 1.2, t = 2;
        Thread1 thread1 = new Thread1(K, a, t);
        Thread2 thread2 = new Thread2(a, t);
        thread1.start(); // Start thread1
        thread2.start(); // Start thread2
        try { // Wait for both threads to complete
            thread1.join();
            thread2.join();
            System.out.println("Value of expression: " +
                               thread1.getValue()*thread2.getValue());
        }
        catch (InterruptedException e) {
            System.out.println("A thread didn't finish!");
        }
    }
}
```

Useful Tips

- **Give names** to your Threads, so you can **distinguish** between them when needed!

```
Thread thread1 = new Thread("Thread 1");
```

```
Thread thread2 = new Thread("Thread 2");
```

```
...
```

```
System.out.println("Run by: " + getName());
```

- **Thread.currentThread()** can be handy:

```
Thread thread = Thread.currentThread();
```

```
String threadName =
```

```
Thread.currentThread().getName();
```


Common Pitfalls

- In earlier versions of Java, there was a **stop()** method which would stop an executing thread. 
- Although this method still exists, it's been **deprecated** and shouldn't be used because it can make a program behave in an unexpected way.
- A thread **terminates** when its **run()** method completes
 - either by returning normally, or
 - by throwing an unchecked exception (RuntimeException, Error or one of their subclasses)
- A multi-threaded program will continue to run until its last (non-daemon) thread terminates

Common Pitfalls

- Threads are **not restartable**: invoking `start()` more than once results in an **`InvalidThreadStateException`**.



Calling **`run()`** instead of **`start()`**:

- When calling the **`run()`** method we don't take the advantage of multithreading: it behaves just like a normal method
- When calling a **`start()`** method, then a **new separate thread** is being allocated to the execution of run method.
- If there are more than 2 threads, called with **`start()`**, then their run method is being executed by separate threads and these threads run simultaneously.
- Illustrative example with calling `run()` instead of `start()`



Calling **run()** method

WRONG

```
public class RunMethodExample implements Runnable{
    public void run(){
        for(int i=1;i<=3;i++){
            try{
                Thread.sleep(1000);
            }catch (InterruptedException ie){
                ie.printStackTrace();
            }
            System.out.println(i);
        }
    }
    public static void main(String args[]){
        Thread th1 = new Thread(new RunMethodExample(), "th1");
        Thread th2 = new Thread(new RunMethodExample(), "th2");
        th1.run();
        th2.run();
    }
}
```



Output

1
2
3
1
2
3



Calling **start()** method

OK!

```
public class RunMethodExample2 implements Runnable {  
    public void run(){  
        for(int i=1;i<=3;i++){  
            try{  
                Thread.sleep(1000);  
            }  
            catch (InterruptedException ie) {  
                ie.printStackTrace();  
            }  
            System.out.println(i);  
        }  
    }  
    public static void main(String args[]) {  
        Thread th1 = new Thread(new RunMethodExample(), "th1");  
        Thread th2 = new Thread(new RunMethodExample(), "th2");  
        th1.start(); th2.start();  
    }  
}
```



Output

1
1
2
2
3
3



Additional Slides

- UML Refresher
 - Difference between class and interface?
- Threads vs. Processes (OS view)
- Others ways of creating and starting Java threads
 - Using anonymous classes
 - Using Lambda expressions
- Acknowledgements and resources for further reading

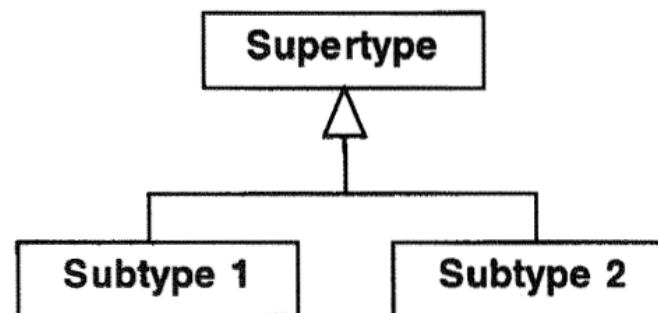
UML Refresher

- UML = Unified Modeling Language
- A visual paradigm to represent OOP classes

Class

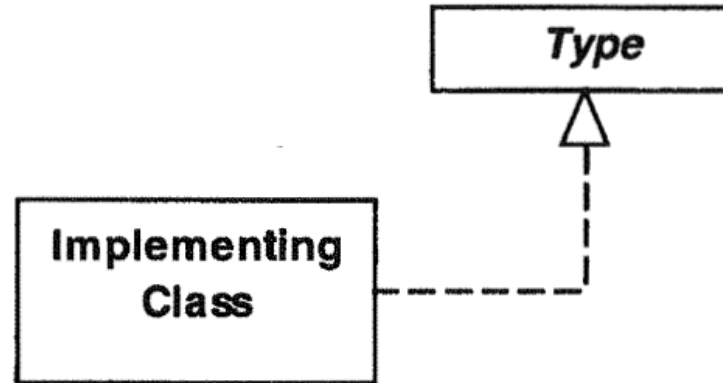


Generalization

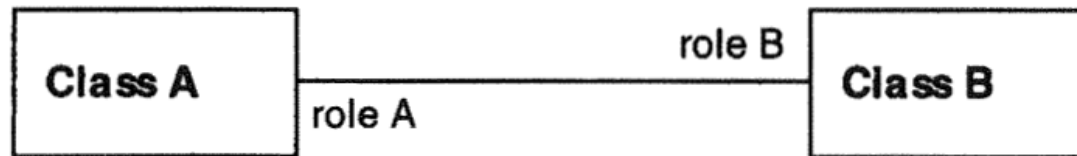


UML Refresher

Interface

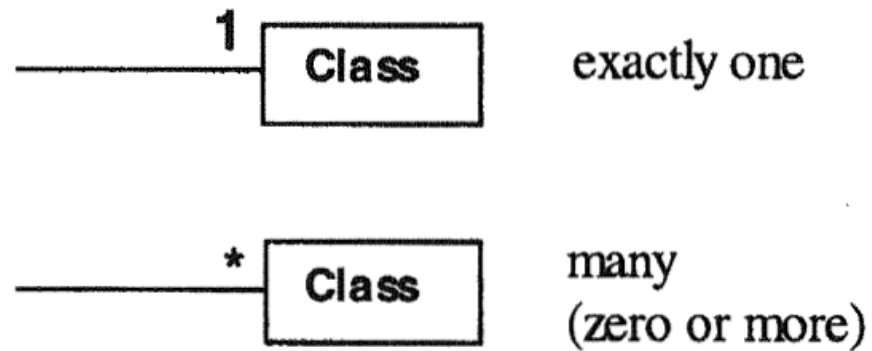


Association

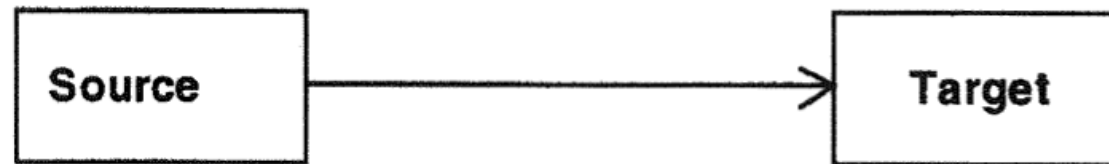


UML Refresher

Multiplicities



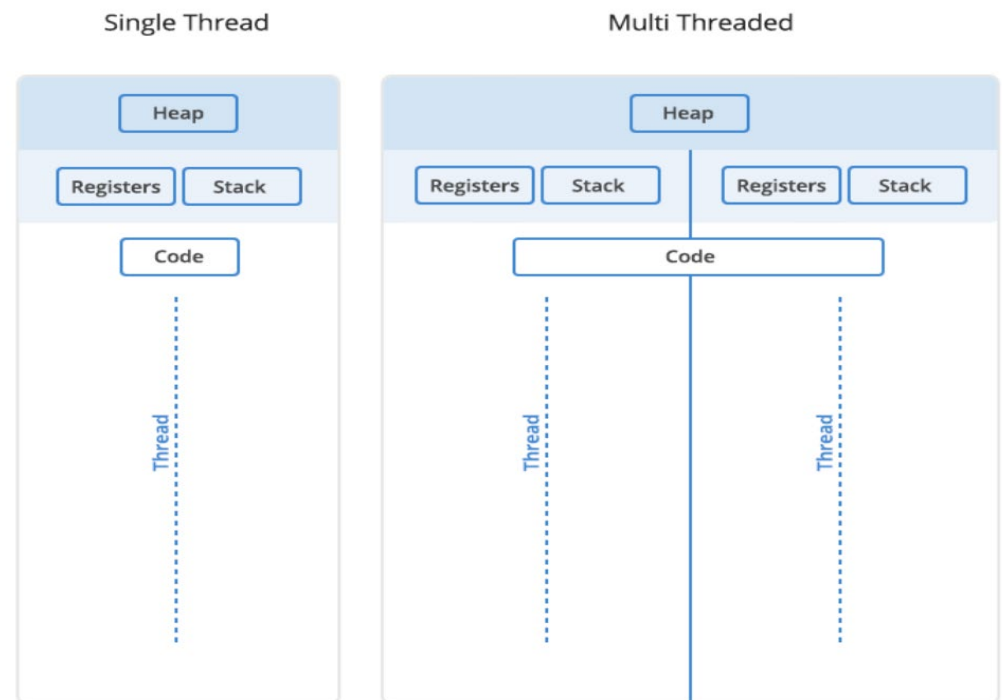
Navigability



Program, Process, Thread (in OS)

- **Program** contains a set of instructions designed to complete a specific task.
- **Process** is an instance of an executing program.
- **Process** means a program is in execution, whereas **thread** means a segment of a process.

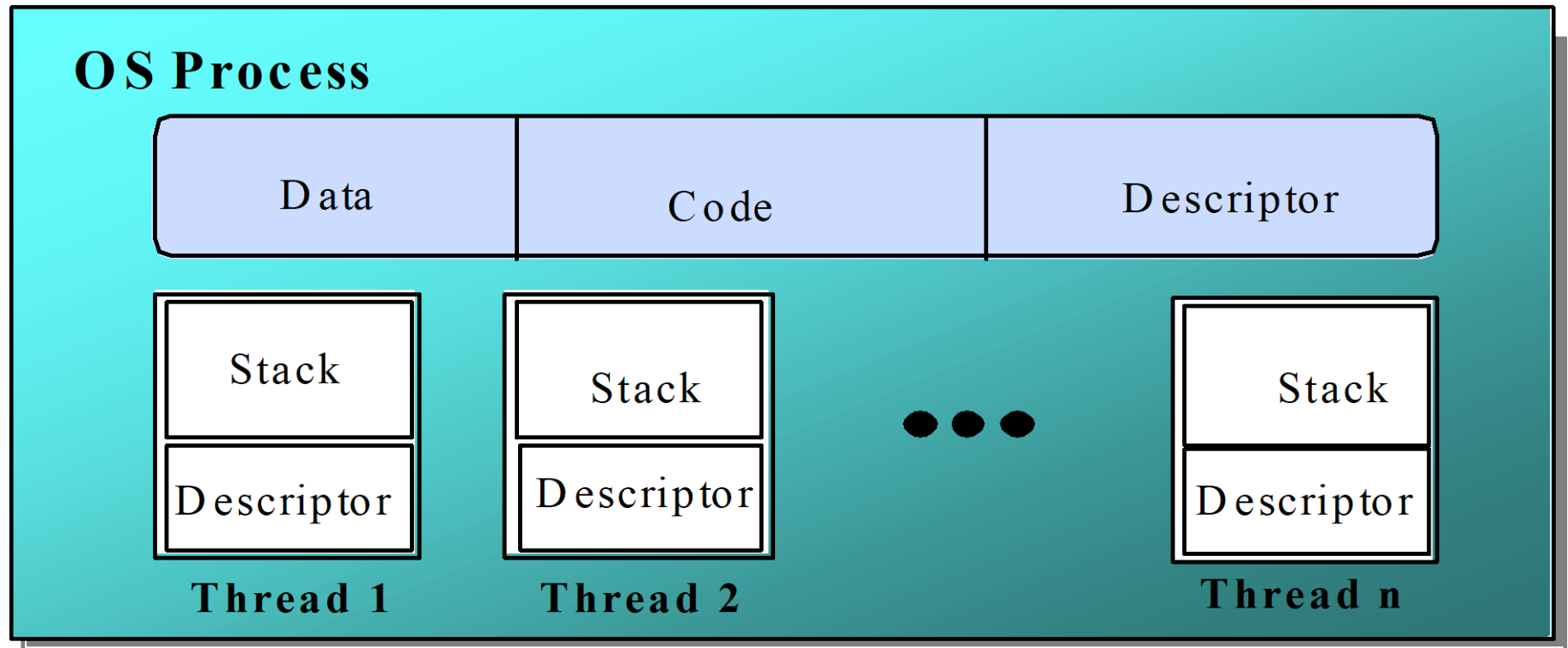
Example: If you're running multiple tabs in Google Chrome or multiple instances of MS Word, each one of them may run as a separate process



Processes vs Threads

- A **process** in an OS is a unit of resource allocation both for CPU time and for memory.
 - Represented by its code, data and the state of the machine registers.
 - Has its own address space which includes also the stack (containing global variables and local variables) and heap.
- A process can have **one or more threads** of control.
- **Threads** are sometimes called *lightweight processes*.
- Both processes and threads provide an execution environment, but creating a new thread requires fewer resources than creating a new process.
- Threads exist within a process — every process has at least one.
- Threads share the process's resources, including memory and open files. This makes for efficient, but potentially problematic, communication.

Implementing Processes: OS View



A **(heavyweight) process** in an operating system is represented by its code, data and the state of the machine registers, given in a descriptor. In order to support multiple (lightweight) **threads of control**, it has multiple stacks, one for each thread.

Creating and Starting Threads

- Here you can see other ways to create threads in Java:
<http://tutorials.jenkov.com/java-concurrency/creating-and-starting-threads.html>
- All of them are based Thread class or Runnable interface:
 1. Extending **Thread** class and overriding its **run()** method
 - can be also with anonymous implementation
 2. Implementing **Runnable** interface and its **run()** method
 - can be also with anonymous implementation
 3. Using a Java **Lambda** implementation of **Runnable**
 - Lambda expression: a function which can be created without belonging to any class; can be passed around as if it was an object and executed on demand.
 - Lambda expressions are Java's first step into functional programming.

1a. Extending Thread class

- First, create a subclass of Thread and override the run() method.

```
public class MyThread extends Thread {  
    public void run() {  
        System.out.println("MyThread running");  
    }  
}
```

- To create and start the above thread

Similar to slide 7!

```
MyThread myThread1 = new MyThread();  
myThread1.start();
```

1b. Extending Thread class

- Using an **anonymous** subclass of Thread

```
Thread myThread2 = new Thread() {  
    public void run() {  
        System.out.println("Thread Running");  
    }  
}  
  
myThread2.start();
```

2a. Implementing Runnable interface

Creating a Java class that implements the Runnable interface:

```
public class MyRunnable implements Runnable {  
    public void run() {  
        System.out.println("MyRunnable running");  
    }  
}
```

Similar to slide 8!

```
Thread thread3 = new Thread(new MyRunnable());  
thread3.start();
```

2b. Implementing Runnable interface

With an **anonymous** implementation of Runnable

```
Runnable myRunnable2 = new Runnable() {  
    public void run() {  
        System.out.println("Runnable running");  
    }  
}
```

```
Thread thread4 = new Thread(myRunnable2);  
thread4.start();
```


3. Using a Java Lambda implementation of Runnable

Creating a Java Lambda implementation of the Runnable interface:

```
Runnable runnable = () -> {  
    System.out.println("Lambda Runnable  
        running");  
};
```

Starting the thread:

```
Thread thread5 = new Thread(runnable);  
thread5.start();
```

Learn more about Java Lambda expressions:

https://www.w3schools.com/java/java_lambda.asp

Acknowledgments (and resources for further reading)

Books

- J. Fernandez Gonzalez (2017), Java 9 Concurrency Cookbook (2nd edition), Packt Publishing / Java 7 Concurrency Cookbook [link](#)
- B. Wittman, T. Korb, A. Mathur (2013), Start Concurrent: [An Introduction to Problem Solving in Java with a Focus on Concurrency](#), Purdue University Press. Chapter 13:
<https://start-concurrent.github.io/chunked/chap12.html#ch13-concurrency>
- Magee & Kramer (2006). Concurrency: State Models and Java Programs (2nd edition), John Wiley. Online slides <https://www.doc.ic.ac.uk/~jnm/book/>

Online tutorials

- <https://docs.oracle.com/javase/tutorial/essential/concurrency/>
- Jakob Jenkov. Java Concurrency Utilities <http://tutorials.jenkov.com/java-concurrency/index.html>
- <https://beginnersbook.com/2015/03/why-dont-we-call-run-method-directly-why-call-start-method/>